

WORKED EXAMPLE

Start from a truth table

A small study example applying Lecture 2, pages 61-72; not the slide's five-input data.

Inputs A, B and C are Boolean. Output F must be 1 on exactly four input rows.

ABC	F
000	0
001	1
010	0
011	1
100	1
101	1
110	0
111	0

A literal is one input or its negation. NOT A means $1 - A$ for a Boolean input.

WORKED EXAMPLE

Write one AND term per true row

DNF means an OR of AND terms. The bar over a letter means NOT.

$$001 : t_1 = \bar{A} \bar{B} C$$

$$011 : t_2 = \bar{A} B C$$

$$100 : t_3 = A \bar{B} \bar{C}$$

$$101 : t_4 = A \bar{B} C$$

$$F = t_1 \vee t_2 \vee t_3 \vee t_4$$

Juxtaposed literals mean AND, not a new variable. A term tests all three bits; the final OR accepts any matching true row.

WORKED EXAMPLE

Turn a row detector into a neuron

For row 001, all three matching literal values must be 1.

$$t_1 = H((1 - A) + (1 - B) + C - 2.5)$$

$$= H(2 - A - B + C - 2.5) = H(-A - B + C - 0.5)$$

Three matches sum to 3 and give score +0.5. Any mismatch lowers that sum to at most 2, so the score is at most -0.5.

$$(W_A, W_B, W_C) = (-1, -1, 1), \quad b = -0.5, \quad T = 0.5$$

Negative weights encode the negated literals directly. No separate NOT neuron is needed.

We use $H(s) = 1$ when $s \geq 0$. Half-integer thresholds avoid ties on Boolean inputs.

WORKED EXAMPLE

The four hidden row detectors

Expand the complements in each matching-literal test in exactly the same way.

$$t_1 = H(-A - B + C - 0.5)$$

$$t_2 = H((1 - A) + B + C - 2.5) = H(-A + B + C - 1.5)$$

$$t_3 = H(A + (1 - B) + (1 - C) - 2.5) = H(A - B - C - 0.5)$$

$$t_4 = H(A + (1 - B) + C - 2.5) = H(A - B + C - 1.5)$$

$$F = H(t_1 + t_2 + t_3 + t_4 - 0.5)$$

The output uses weights (1, 1, 1, 1) and threshold 0.5: at least one detector must fire.

WORKED EXAMPLE

Work one row through every neuron

For input 001, only its own row detector should fire.

$$s_{001} = -0 - 0 + 1 - 0.5 = 0.5 \Rightarrow t_1 = 1$$

$$s_{011} = -0 + 0 + 1 - 1.5 = -0.5 \Rightarrow t_2 = 0$$

$$s_{100} = 0 - 0 - 1 - 0.5 = -1.5 \Rightarrow t_3 = 0$$

$$s_{101} = 0 - 0 + 1 - 1.5 = -0.5 \Rightarrow t_4 = 0$$

$$F = H(1 + 0 + 0 + 0 - 0.5) = H(0.5) = 1$$

WORKED EXAMPLE

Verify every canonical-network row

Scores are before H; hidden is the four-bit vector after H in detector order.

ABC	s001	s011	s100	s101	hidden	output
000	-0.5	-1.5	-0.5	-1.5	0000	0
001	0.5	-0.5	-1.5	-0.5	1000	1
010	-1.5	-0.5	-1.5	-2.5	0000	0
011	-0.5	0.5	-2.5	-1.5	0100	1
100	-1.5	-2.5	0.5	-0.5	0010	1
101	-0.5	-1.5	-0.5	0.5	0001	1
110	-2.5	-1.5	-0.5	-1.5	0000	0
111	-1.5	-0.5	-1.5	-0.5	0000	0

Detector order: 001, 011, 100, 101. A positive score becomes 1; every negative score becomes 0.

In each true row, hidden sums to 1 and output score is +0.5. In every false row, hidden sums to 0 and output score is -0.5.

WORKED EXAMPLE

Arrange the same table as a K-map

Columns use Gray order: 00, 01, 11, 10. Adjacent columns differ by one bit.

A / BC	00	01	11	10
0	0	1	1	0
1	1	1	0	0

Top pair 001/011: $A = 0$ and $C = 1$ stay fixed; B changes, so remove B .

Bottom pair 100/101: $A = 1$ and $B = 0$ stay fixed; C changes, so remove C .

Groups have power-of-two size. Adjacent edge cells may wrap around; diagonal cells are not adjacent.

WORKED EXAMPLE

The reduced expression needs two detectors

Grouping removed tests whose input can change without changing the output.

$$F = (\bar{A} \wedge C) \vee (A \wedge \bar{B})$$

$$h_1 = H((1 - A) + C - 1.5) = H(-A + C - 0.5)$$

$$h_2 = H(A + (1 - B) - 1.5) = H(A - B - 0.5)$$

$$F = H(h_1 + h_2 - 0.5)$$

h_1 ignores B ; h_2 ignores C . Each detector now checks two literals rather than three.

This reduces the number of hidden neurons from four to two without changing the truth table.

WORKED EXAMPLE

Verify reduced network: rows 1-2

Substitute inputs, threshold both hidden scores, then threshold their sum.

$$000 : \quad h_1 = H(-0 + 0 - 0.5) = H(-0.5) = 0$$

$$h_2 = H(0 - 0 - 0.5) = H(-0.5) = 0$$

$$F = H(0 + 0 - 0.5) = H(-0.5) = 0$$

$$001 : \quad h_1 = H(-0 + 1 - 0.5) = H(0.5) = 1$$

$$h_2 = H(0 - 0 - 0.5) = H(-0.5) = 0$$

$$F = H(1 + 0 - 0.5) = H(0.5) = 1$$

WORKED EXAMPLE

Verify reduced network: rows 3-4

Substitute inputs, threshold both hidden scores, then threshold their sum.

$$010: \quad h_1 = H(-0 + 0 - 0.5) = H(-0.5) = 0$$

$$h_2 = H(0 - 1 - 0.5) = H(-1.5) = 0$$

$$F = H(0 + 0 - 0.5) = H(-0.5) = 0$$

$$011: \quad h_1 = H(-0 + 1 - 0.5) = H(0.5) = 1$$

$$h_2 = H(0 - 1 - 0.5) = H(-1.5) = 0$$

$$F = H(1 + 0 - 0.5) = H(0.5) = 1$$

WORKED EXAMPLE

Verify reduced network: rows 5-6

Substitute inputs, threshold both hidden scores, then threshold their sum.

$$100 : \quad h_1 = H(-1 + 0 - 0.5) = H(-1.5) = 0$$

$$h_2 = H(1 - 0 - 0.5) = H(0.5) = 1$$

$$F = H(0 + 1 - 0.5) = H(0.5) = 1$$

$$101 : \quad h_1 = H(-1 + 1 - 0.5) = H(-0.5) = 0$$

$$h_2 = H(1 - 0 - 0.5) = H(0.5) = 1$$

$$F = H(0 + 1 - 0.5) = H(0.5) = 1$$

WORKED EXAMPLE

Verify reduced network: rows 7-8

Substitute inputs, threshold both hidden scores, then threshold their sum.

$$110: \quad h_1 = H(-1 + 0 - 0.5) = H(-1.5) = 0$$

$$h_2 = H(1 - 1 - 0.5) = H(-0.5) = 0$$

$$F = H(0 + 0 - 0.5) = H(-0.5) = 0$$

$$111: \quad h_1 = H(-1 + 1 - 0.5) = H(-0.5) = 0$$

$$h_2 = H(1 - 1 - 0.5) = H(-0.5) = 0$$

$$F = H(0 + 0 - 0.5) = H(-0.5) = 0$$

WORKED EXAMPLE

Count what the simplification saved

A stored zero weight still counts as a parameter of a dense layer.

$$\text{canonical : } 3(4) + 4 + 4(1) + 1 = 21$$

Four hidden neurons each have three input weights and a bias; the output has four weights and one bias.

$$\text{reduced dense : } 3(2) + 2 + 2(1) + 1 = 11$$

$$\text{reduced sparse : } 2 + 2 + 2 + 3 = 9$$

The sparse version omits the unused B-to-h1 and C-to-h2 edges: six present weights plus three biases. Counts depend on how connections are stored.

INDEPENDENT PRACTICE

Your turn: a different truth table

Set $F = 1$ for 010, 011, 101 and 111; set $F = 0$ elsewhere.

-
1. Place all eight rows into a K-map with columns $BC = 00, 01, 11, 10$.
 2. Group adjacent ones and write a reduced OR-of-ANDs expression.
 3. Convert each term into a hidden neuron, then add an OR output.
 4. Check all eight rows against the requested truth table.

Use the same H convention and half-integer thresholds.

WORKED ANSWER

Practice answer: group and translate

Each group removes the input that changes inside that group.

A / BC	00	01	11	10
0	0	0	1	1
1	0	1	1	0

Top pair 010/011 keeps $A = 0, B = 1$: NOT A AND B. Bottom pair 101/111 keeps $A = 1, C = 1$: A AND C.

$$F = (\bar{A} \wedge B) \vee (A \wedge C)$$

$$h_1 = H(-A + B - 0.5), \quad h_2 = H(A + C - 1.5)$$

$$F = H(h_1 + h_2 - 0.5)$$

WORKED ANSWER

Practice answer: all eight rows

The output applies $H(h1 + h2 - 0.5)$ to each row.

ABC	$s1=-A+B-.5$	$s2=A+C-1.5$	$h1,h2$	F
000	-0.5	-1.5	0,0	0
001	-0.5	-0.5	0,0	0
010	0.5	-1.5	1,0	1
011	0.5	-0.5	1,0	1
100	-1.5	-0.5	0,0	0
101	-1.5	0.5	0,1	1
110	-0.5	-0.5	0,0	0
111	-0.5	0.5	0,1	1

Example 010: $h1 = H(-0 + 1 - 0.5) = 1$; $h2 = H(0 + 0 - 1.5) = 0$; $F = H(1 + 0 - 0.5) = 1$.

Avoid: ordinary binary column order in the map; diagonal grouping; forgetting to adjust the bias when expanding NOT literals.